Echoplex Tape Delay Model

```
% This code implements an Echoplex tape delay model based on the work of Steinunn
Arnardottir,
% Jonathan S. Abel, & Julius O. Smith,  “A digital model of the echoplex tape
delay,” in
% Audio Engineering Society Convention 125, Oct 2008.
% https://secure.aes.org/forum/pubs/conventions/?elib=14800

% For more information and the downloadable plugin, please visit
% https://michaelzajner.com/research/echoplex/

% The model includes interpolated write, anti-aliasing filtering, leaky integrator,
% and additional irregularities such as capstan drift and pinch wheel drift.
```

Section 1: Initialization and Parameters Setup

```
% Adjustable Parameters
% Many parameters are approximations based on the materials provided in the
% article.
% clear all;

sampleRate = 48000; % Sampling rate in Hz
initialDelayMs = 300; % Initial delay in milliseconds (can be adjusted to control
delay time)
combFeedbackGain = 0.1; % Comb filter feedback gain (adjustable to control comb
filtering effect)
delayFeedbackGain = 0.95; % Feedback/repeats control (adjustable to control repeats
amount)
leakCoefficient = 0.95; % Coefficient for leaky integrator (adjustable for natural
decay effect)
dryWetMix = 0.75; % Dry/Wet Mix Ratio (0 = All Dry, 1 = All Wet)
saturationAmount = 0.5; % Amount of saturation applied (0 = No saturation, 1 = Full
saturation)
modulationAmount = 0.1; % Modulation control of capstan & pinch wheel (0 = No
modulation, 1 = Full modulation)

% Simulate moving delay time
delaySweepRangeMs = 100; % Maximum variation for sweeping delay in milliseconds
sweepFrequency = 10; % Frequency of the sweeping delay in Hz

% Convert delay time to samples
initialDelaySamples = round(initialDelayMs * (sampleRate / 1000)); % Convert delay
to samples

% Create UI controls for parameters
%{
fig = uifigure('Name', 'Echoplex Tape Delay Controls');
```

```
initialDelaySlider = uislider(fig, 'Position', [100, 300, 300, 3], 'Limits', [50,
800], 'Value', initialDelayMs);
initialDelaySlider.ValueChangedFcn = @(src, event) assignin('base',
'initialDelayMs', src.Value);
initialDelaySlider.Tooltip = 'Delay (ms)';

delayFeedbackSlider = uislider(fig, 'Position', [100, 250, 300, 3], 'Limits', [0,
1.5], 'Value', delayFeedbackGain);
delayFeedbackSlider.ValueChangedFcn = @(src, event) assignin('base',
'delayFeedbackGain', src.Value);
delayFeedbackSlider.Tooltip = 'Repeats';

dryWetMixSlider = uislider(fig, 'Position', [100, 200, 300, 3], 'Limits', [0, 1],
'Value', dryWetMix);
dryWetMixSlider.ValueChangedFcn = @(src, event) assignin('base', 'Mix', src.Value);
dryWetMixSlider.Tooltip = 'Dry/Wet Mix Ratio';

modulationAmountSlider = uislider(fig, 'Position', [100, 150, 300, 3], 'Limits',
[0, 0.05], 'Value', modulationAmount);
modulationAmountSlider.ValueChangedFcn = @(src, event) assignin('base',
'modulationAmount', src.Value);
modulationAmountSlider.Tooltip = 'Modulation Amount';
%}
```

Section 2: Load and Prepare Audio File

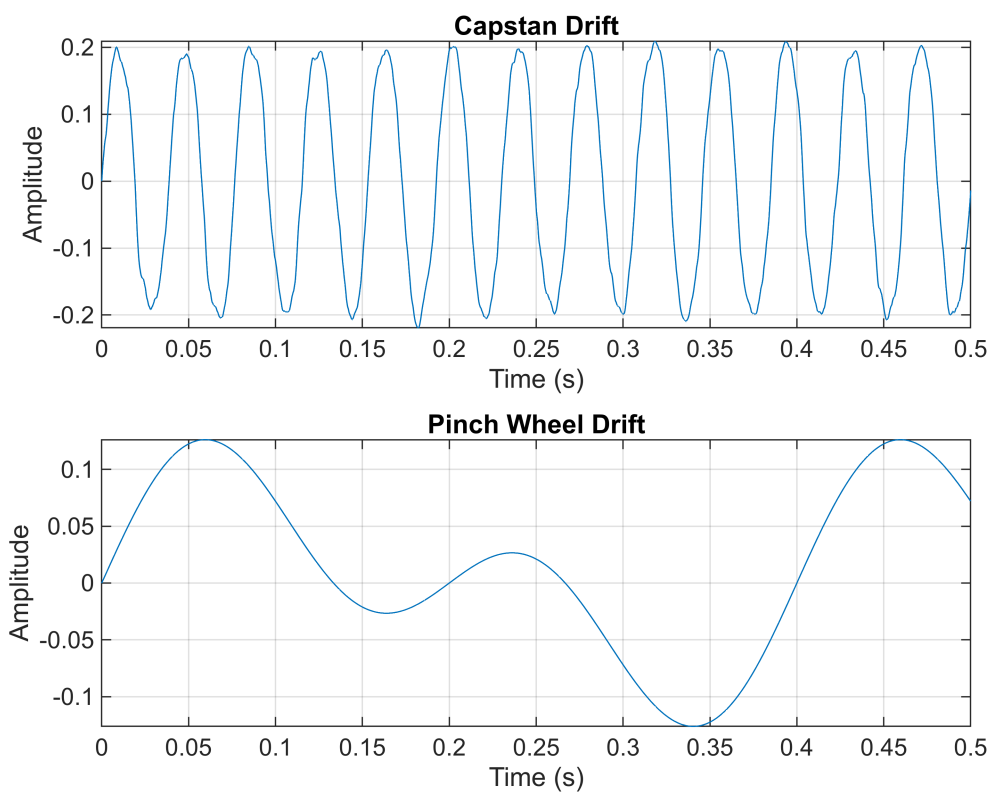
```
% For testing: Load an audio file (mono channel), and set up parameters.
% [inputSignal, sampleRate] = audioread('Test.wav'); % Replace with your audio file
% inputSignal = inputSignal(:, 1)'; % Use the first channel if stereo, transpose to
a row vector

% For testing: Generate white noise
% testDuration = 5; % Duration of the test signal in seconds
% whiteNoiseInput = randn(1, sampleRate * testDuration); % Generate white noise

% For testing: Generate an impulse response
% testDuration = 1; % Duration of the test signal in samples
% impulseInput = zeros(1, sampleRate);
% impulseInput(1) = 1; % Set the first sample to 1 to create an impulse

% For testing: % Generate sine sweep
% sineSweepInput = chirp(timeVector, 20, timeVector(end), 20000); % Frequency sweep
from 20 Hz to 20 kHz

% For testing: % Generate sinewave
% sineFreq = 100; % Frequency in Hz
% sineWaveInput = sin(2 * pi * sineFreq * (0:1/sampleRate:1)); % 1 second sine wave

% Replace the 'inputSignal' with the impulse or white noise for testing
% inputSignal = whiteNoiseInput;
```

```
% inputSignal = impulseInput;
% inputSignal = sineSweepInput;
% inputSignal = sineWaveInput;

% Time vector based on audio length
timeVector = (0:length(inputSignal)-1) / sampleRate;

%% Section 2.1: Sweeping Delay Modulation Setup
% Generate sweeping modulation for delay time
sweepModulation = sawtooth(2 * pi * sweepFrequency * timeVector, 0.5) *
(delaySweepRangeMs / 2);

% Create a sharp drop lasting 30 milliseconds
sharpDropDurationSamples = round(30 * (sampleRate / 1000));
sharpDrop = linspace(1, 0, sharpDropDurationSamples);
sweepModulation = [sweepModulation(1:end-sharpDropDurationSamples), sharpDrop];

sweepingDelaySamples = initialDelaySamples + round((sweepModulation * (sampleRate /
1000)));
```

Section 3: Capstan Pinch Wheel, and Noise Drift Generation

```
% Generate various drift components affecting the tape delay system.

% Capstan Drift (Sinusoidal)
capstanFrequency = 26; % Frequency for capstan drift in Hz
capstanAmplitudeSec = 0.00005; % Amplitude of the capstan drift
capstanPulse = sin(2 * pi * capstanFrequency * timeVector); % Generate sinusoidal
signal
phaseNoiseCutoff = 100; % Cutoff frequency for phase noise filter in Hz
[b, a] = butter(2, phaseNoiseCutoff / (sampleRate / 2)); % Butterworth IIR filter
(2nd order)
whiteNoise = randn(1, length(timeVector)); % Generate white noise
filteredPhaseNoise = filter(b, a, whiteNoise); % Filter the noise to add drift

combinedPhase = capstanPulse + filteredPhaseNoise; % Combined phase signal
capstanDrift = tanh(sin(combinedPhase) * capstanAmplitudeSec * sampleRate *
modulationAmount); % Capstan drift calculation with limiting

% Pinch Wheel Drift
pinchWheelFreq1 = 2.5; % First pinch wheel irregularity in Hz
pinchWheelFreq2 = 5.0; % Second pinch wheel irregularity in Hz
pinchWheelPulse1 = sin(2 * pi * pinchWheelFreq1 * timeVector);
pinchWheelPulse2 = sin(2 * pi * pinchWheelFreq2 * timeVector);
pinchWheelDrift = tanh((pinchWheelPulse1 + pinchWheelPulse2) * 0.00005 *
initialDelayMs / 1000 * sampleRate * modulationAmount); % Pinch wheel drift
calculation with limiting

% Noise
```

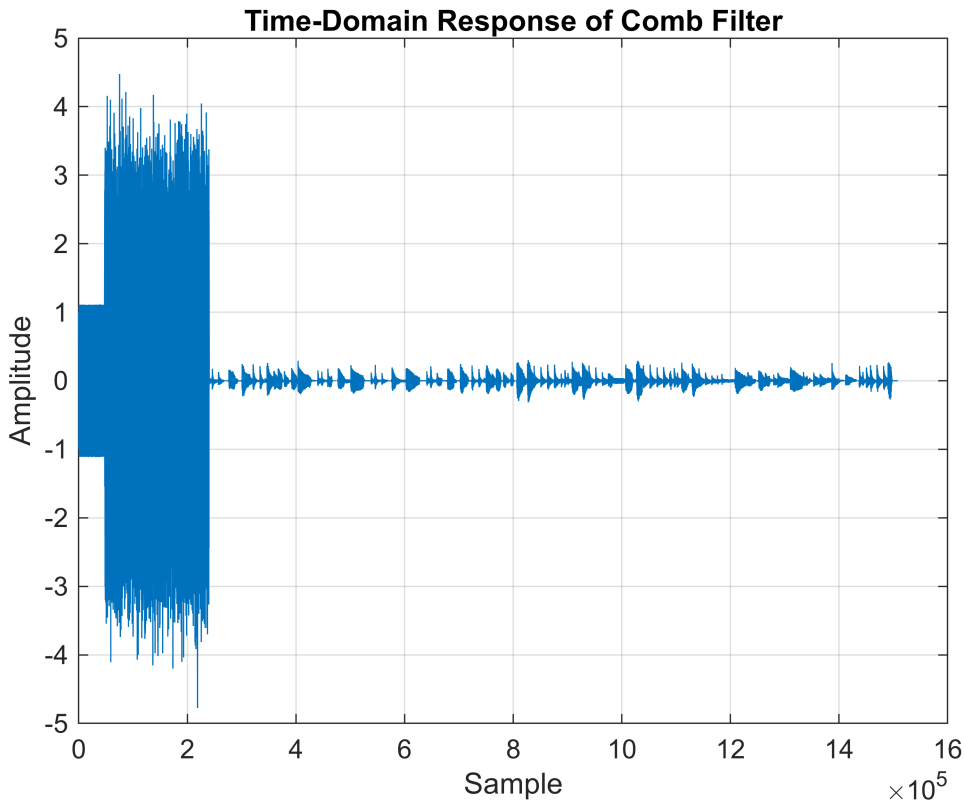
```
% Add noise to simulate mechanical imperfections.
noiseAmplitude = 0.00075 * (initialDelayMs / 1000);
mechanicalNoiseSignal = noiseAmplitude * whiteNoise;
filteredMechanicalNoise = filter(b, a, mechanicalNoiseSignal);

%% Plot Capstan and Pinch Wheel Drift
figure;
subplot(2, 1, 1);
plot(timeVector, capstanDrift);
xlim([0 0.5]); % Focus on the first 0.5 seconds
title('Capstan Drift');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(2, 1, 2);
plot(timeVector, pinchWheelDrift);
xlim([0 0.5]); % Focus on the first 0.5 seconds
title('Pinch Wheel Drift');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```



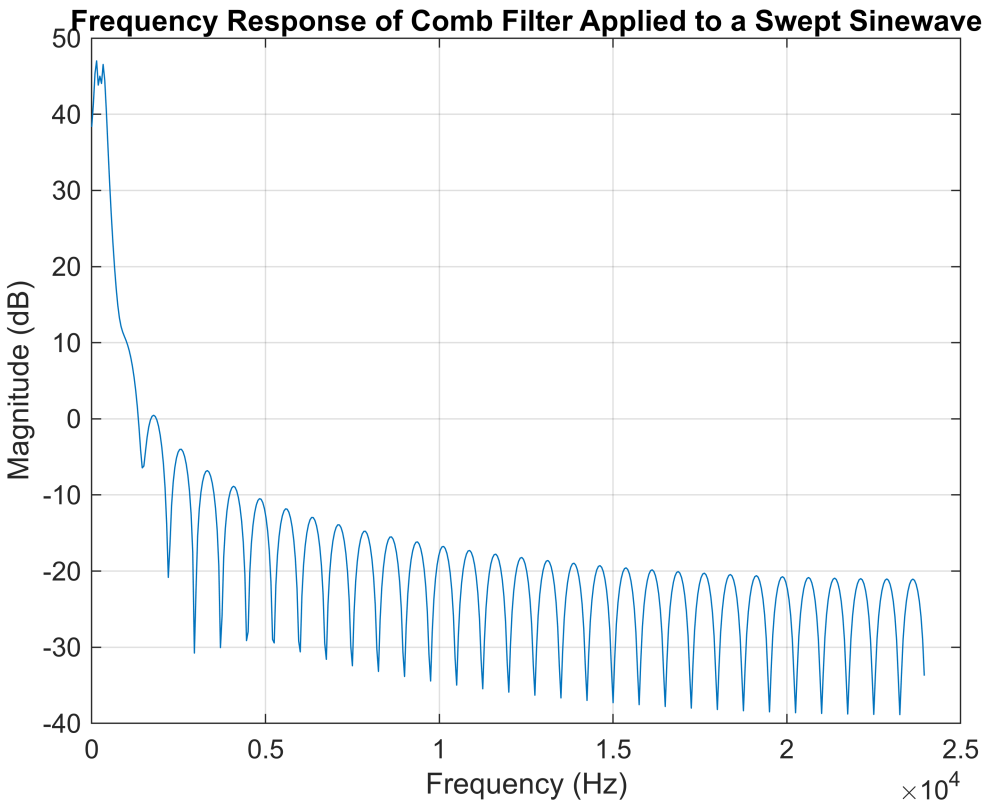
Section 4: Comb Filter

```
% Parameters for Comb Filter
```

```
initialCombDelay = 0.02 * sampleRate; % Base comb delay length (e.g., 20 ms)
combBufferLength = round(initialCombDelay) + 1; % Buffer length for comb filter
combBuffer = zeros(1, combBufferLength); % Initialize comb buffer
combWritePointer = 1; % Write pointer for comb filter

% Update comb filter delay time dynamically
for n = 1:length(inputSignal)
    % Update comb filter delay time based on delay handle position
    currentDelayHandle = sin(2 * pi * sweepFrequency * (n / sampleRate)); % Example
modulation
    combDelaySamples = round(initialCombDelay + modulationAmount *
currentDelayHandle);

    % Ensure delay time is bounded
    combDelaySamples = max(1, min(combDelaySamples, combBufferLength - 1));

    % Retrieve delayed value from comb buffer
    combReadPointer = combWritePointer - combDelaySamples;
    if combReadPointer <= 0
        combReadPointer = combReadPointer + combBufferLength;
    end

    % Calculate comb filter output
    combDelayedValue = combBuffer(combReadPointer);
    combFilterOutput = inputSignal(n) + combFeedbackGain * combDelayedValue;

    % Update comb buffer
    combBuffer(combWritePointer) = combFilterOutput;
    combWritePointer = combWritePointer + 1;
    if combWritePointer > combBufferLength
        combWritePointer = 1; % Wrap around buffer
    end

    % Store the comb filter output for analysis
    combOutputSignal(n) = combFilterOutput;
end

% Plot the time-domain response of the comb filter
figure;
plot(combOutputSignal);
xlabel('Sample');
ylabel('Amplitude');
title('Time-Domain Response of Comb Filter');
grid on;
```




```
% Frequency response of comb filter
fftCombOutput = abs(fft(combOutputSignal, 1024));
freqAxis = (0:511) * (sampleRate / 1024);
figure;
plot(freqAxis, 20*log10(fftCombOutput(1:512)));
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Frequency Response of Comb Filter Applied to a Swept Sinewave');
grid on;
```



Section 5: Anti-Aliasing Filters

```
% Anti-Aliasing Filter Setup
% Implement an FIR filter for anti-aliasing to reduce high-frequency components
antiAliasFilterOrder = 64; % Order of the anti-aliasing filter
antiAliasCutoffFreq = 20000; % Initial cutoff frequency for anti-aliasing filter
(in Hz)
antiAliasCoeffs = fir1(antiAliasFilterOrder, antiAliasCutoffFreq / (sampleRate /
2)); % Design a low-pass FIR filter

% Sinc Interpolation for Anti-Aliasing
% Parameters for sinc interpolation
desiredDelaySamples = 4.7; % Desired delay in samples
numSteps = length(inputSignal); % Number of time steps
delayLineLength = initialDelaySamples + 50;
delayLineBuffer = zeros(1, delayLineLength);
outputSignal = zeros(1, numSteps); % Output signal

% Initialize pointers
writePointer = 1;
readPointer = writePointer - initialDelaySamples;
if readPointer <= 0
    readPointer = readPointer + delayLineLength; % Wrap around for negative values
```

```
end

% Setup FIR filter for anti-aliasing (using sinc interpolation)
sincFilterLength = 32; % Length of sinc filter (must be even)
normalizedCutoffFreq = 0.5; % Normalized cutoff frequency for sinc interpolation
sincKernel = sinc(normalizedCutoffFreq * (-sincFilterLength/2:sincFilterLength/2));
sincKernel = sincKernel .* hann(length(sincKernel))'; % Apply Hann window for
smoother response
```

Section 6: Main Loop for Delay Processing

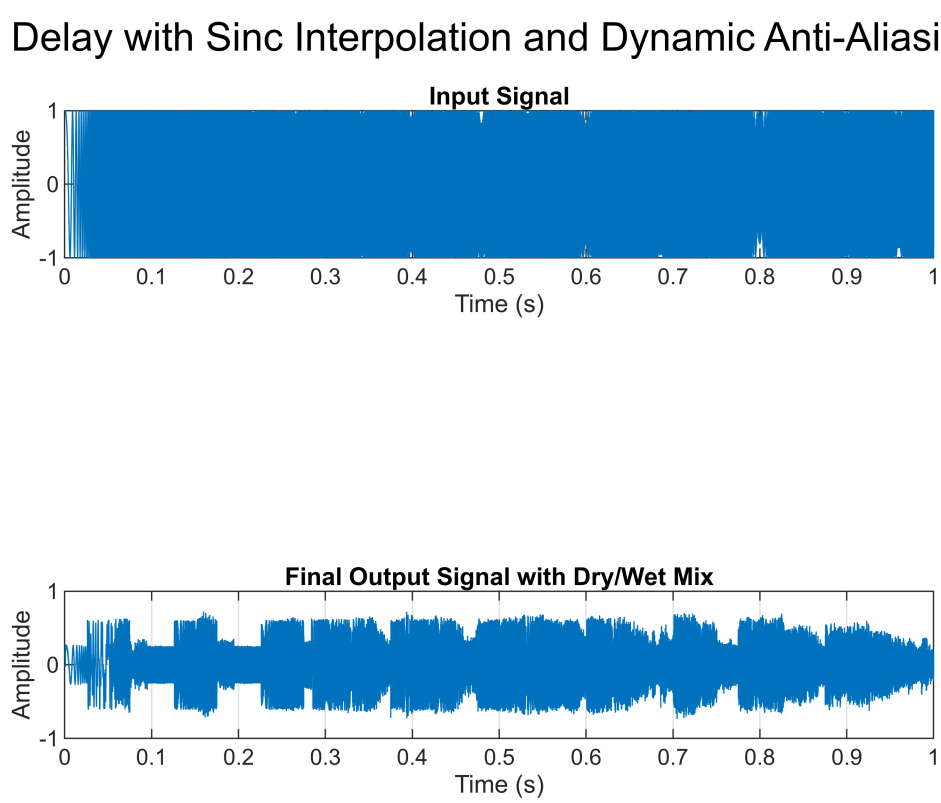
```
% Incorporate the Comb Filter, Leaky Integrator, and Feedback
for n = 1:length(inputSignal)
    % Step 1: Update Read Pointer with Drift
    drift = modulationAmount * (capstanDrift(n) + pinchWheelDrift(n)) +
filteredMechanicalNoise(n);

    % Simulate a sweep in the delay time (remove if desired)
    delaySamples = max(1, sweepingDelaySamples(n)); % Prevent negative or zero delay
    readPointer = writePointer - delaySamples + drift;

    % Without sweeping delay Time
    readPointer = readPointer + 1 + drift;

    % Ensure read pointer wraps around the delay line buffer
    if readPointer > delayLineLength
        readPointer = readPointer - delayLineLength;
    elseif readPointer <= 0
        readPointer = readPointer + delayLineLength;
    end

    % Step 2: Sinc Interpolation for Anti-Aliasing
    interpolatedOutput = 0;
    for k = -sincFilterLength/2:sincFilterLength/2
        currentIndex = floor(readPointer) + k;

        % Handle circular buffer wrapping for the delay line
        if currentIndex <= 0
            currentIndex = currentIndex + delayLineLength;
        elseif currentIndex > delayLineLength
            currentIndex = currentIndex - delayLineLength;
        end

        % Accumulate the weighted samples using the sinc kernel
        interpolatedOutput = interpolatedOutput + delayLineBuffer(currentIndex) *
sincKernel(k + sincFilterLength/2 + 1);
    end

    % Step 3: Apply Feedback Gain (Repeats Control)
```

```
    feedbackSignal = delayFeedbackGain * interpolatedOutput;

    % Step 4: Apply Comb Filter in Feedback Loop
    combDelayedValue = combBuffer(combWritePointer); % Get delayed value from comb
buffer
    combFilterOutput = feedbackSignal + combFeedbackGain * combDelayedValue; %
Apply comb filter feedback
    combBuffer(combWritePointer) = combFilterOutput; % Store in comb buffer for
future use

    % Step 5: Apply Leaky Integrator
    if n == 1
        previousFeedback = 0;
    end
    feedbackSignal = leakCoefficient * combFilterOutput + (1 - leakCoefficient) *
previousFeedback;
    previousFeedback = feedbackSignal;
    saturatedFeedback = saturationAmount * tanh(feedbackSignal);

    % Step 6: Combine Dry and Wet Signal (Dry/Wet Mix)
    drySignal = inputSignal(n);
    wetSignal = saturatedFeedback;
    outputSignal(n) = (1 - dryWetMix) * drySignal + dryWetMix * wetSignal;

    % Step 7: Write Feedback and Current Input Back to Delay Line
    delayLineBuffer(writePointer) = saturatedFeedback + inputSignal(n);

    % Update the read and write pointers for the delay line, handle circular
wrapping
    writePointer = writePointer + 1;
    combWritePointer = combWritePointer + 1;

    if writePointer > delayLineLength
        writePointer = 1;
    end
    if combWritePointer > combBufferLength
        combWritePointer = 1; % Wrap around the buffer
    end
end

% Apply Anti-Aliasing Filter to Output Signal
outputSignal = filter(antiAliasCoeffs, 1, outputSignal); % Apply anti-aliasing
filter to output signal
```

Section 7: Plotting the Results

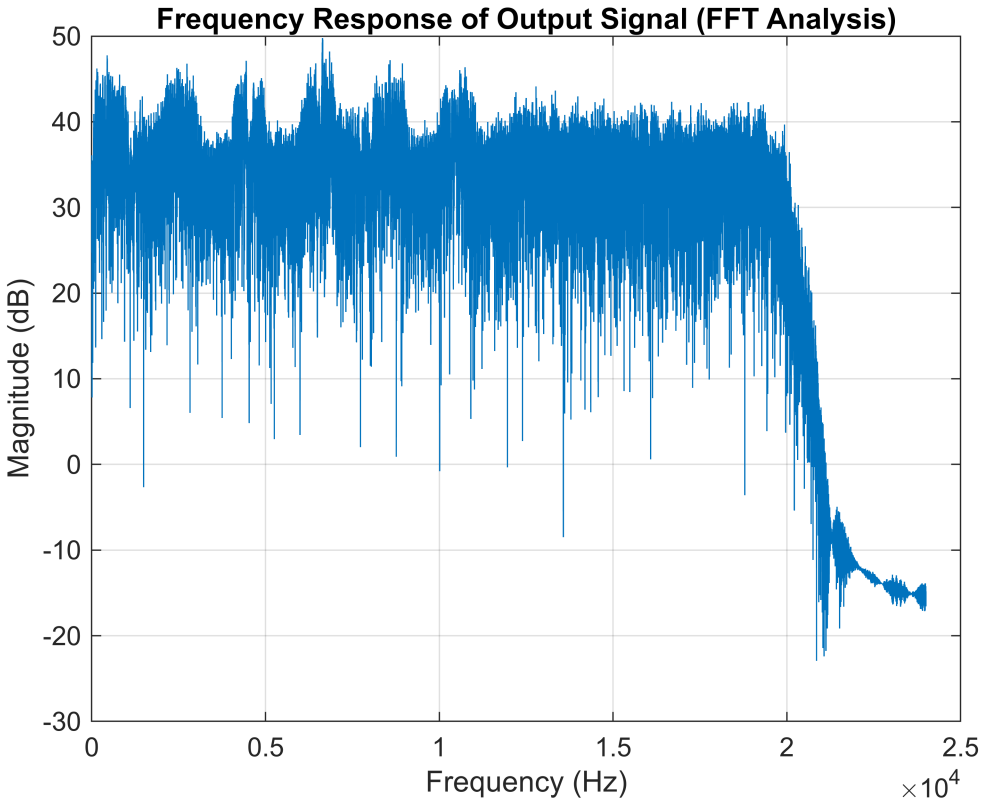
```
% Plot the input signal for reference
figure;
subplot(3,1,1);
```

```
plot(timeVector, inputSignal);
title('Input Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

% Plot the final output signal with dry/wet mix applied
subplot(3,1,3);
plot(timeVector, outputSignal);
title('Final Output Signal with Dry/Wet Mix');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

sgtitle('Echoplex Tape Delay with Sinc Interpolation and Dynamic Anti-Aliasing
Visualization');
```



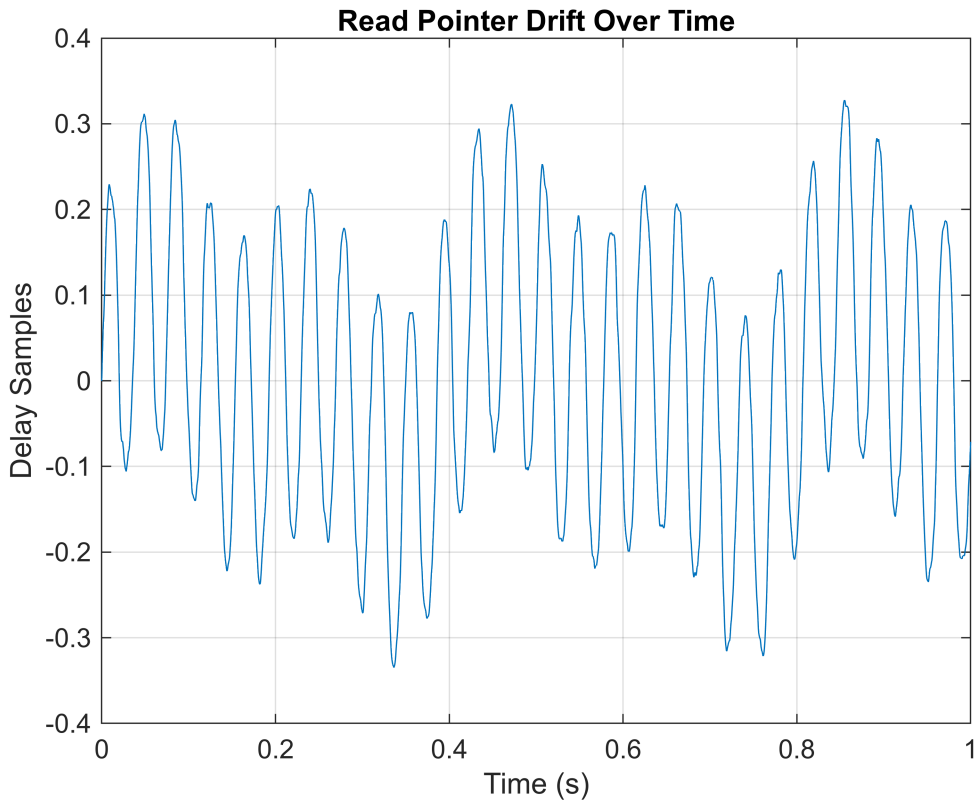

```
% Frequency Response Analysis (FFT)
figure;
outputFFT = abs(fft(outputSignal));
frequencies = linspace(0, sampleRate/2, length(outputFFT)/2 + 1);
plot(frequencies, 20*log10(outputFFT(1:length(frequencies))));
title('Frequency Response of Output Signal (FFT Analysis)');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
```

```
grid on;
```




```
%{
% Plot input vs. output for the sine sweep
figure;
subplot(2, 1, 1);
plot(timeVector, sineSweepInput);
title('Sine Sweep Input');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(2, 1, 2);
plot(timeVector, outputSignal);
title('Sine Sweep Output');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
%}

%{
% Plotting results for the sinc interpolation with impulse response
timeSteps = 0:numSteps-1;

figure;
subplot(3,1,1)
```

```
stem(timeSteps, inputSignal(1:numSteps))
grid
xlabel('Time (samples)');
ylabel('Amplitude');
title('Input Signal (Impulse)');

subplot(3,1,2)
stem(timeSteps, outputSignal)
grid
xlabel('Time (samples)');
ylabel('Amplitude');
title('Sinc Interpolated Output Signal');

subplot(3,1,3)
plot(timeSteps, outputSignal)
grid
xlabel('Time (samples)');
ylabel('Amplitude');
title('Interpolated Output for Debugging');

sgtitle('Sinc Interpolation Debugging for Impulse Response');
%}

% Plot the read pointer drift over time
figure;
plot(timeVector, capstanDrift + pinchWheelDrift);
title('Read Pointer Drift Over Time');
xlabel('Time (s)');
ylabel('Delay Samples');
grid on;
```



Section 7: Output

```
% Normalize and Clip the Output Signal
% Normalize to prevent clipping during audio writing
maxAmplitude = max(abs(outputSignal)); % Find the maximum absolute value
if maxAmplitude > 1
    outputSignal = outputSignal / maxAmplitude; % Normalize to range -1 to 1
end

% Optional: Clip to ensure no value exceeds -1 to 1
outputSignal = max(min(outputSignal, 1), -1);

%% Save the Output Audio File
outputFilename =
['C:\Users\mikez\Documents\MATLAB\Echoplex\AudioExamples\modulation.wav']; % Change
to desired output location
audiowrite(outputFilename, outputSignal, sampleRate); % Save the output signal
fprintf('Processed test output saved to: %s\n', outputFilename);
```

```
Processed test output saved to: C:\Users\mikez\Documents\MATLAB\Echoplex\AudioExamples\modulation.wav
```